SVHN Digit Recognition

CNN Training, Evaluation, API Deployment, and Dockerization

Machine Learning Engineering Project Report

Author: Jerome Jabson

Version: 1.0

Date: July 2026

Table of Contents

List of Figures

List of Tables

1. Executive Summary

This report documents the development and deployment of a convolutional neural network for classifying cropped street-view house-number images from the SVHN dataset.

The project includes data exploration, preprocessing, CNN model development, model evaluation, modular inference code, a FastAPI prediction service, and Docker-based deployment. The completed API accepts an uploaded digit image and returns the predicted class, numeric confidence, human-readable confidence percentage, and the probability assigned to each digit class.

2. Problem Statement

Recognizing handwritten or street-view digits is a fundamental computer vision task with applications in mail sorting, package delivery, navigation systems, utility meter reading, traffic monitoring, and autonomous vehicles. While the problem appears simple to humans, variations in lighting, viewing angles, background clutter, and digit shape make automated recognition challenging.

The Street View House Numbers (SVHN) dataset provides real-world images of house numbers captured from Google Street View. Unlike handwritten digit datasets such as MNIST, SVHN contains naturally occurring images with greater variation in color, scale, orientation, and background complexity. These characteristics make SVHN a more realistic benchmark for evaluating modern computer vision models.

The objective of this project is to design, train, evaluate, and deploy a Convolutional Neural Network (CNN) capable of accurately classifying cropped house-number images into one of the ten digit classes (0–9). Beyond model development, the project demonstrates production-oriented machine learning engineering practices by exposing the trained model through a FastAPI REST API and packaging the complete application in a Docker container for reproducible deployment.

3. Project Objectives

The primary objectives of this project were to design, implement, evaluate, and deploy a deep learning solution for handwritten digit recognition using the Street View House Numbers (SVHN) dataset.

The project objectives are summarized below:

- Develop a Convolutional Neural Network (CNN) capable of classifying digits into one of ten classes (0–9).
- Perform data preprocessing, normalization, and label preparation suitable for deep learning.
- Evaluate model performance using accuracy, precision, recall, F1-score, and confusion matrices.
- Separate model inference from training by implementing a reusable inference module.
- Develop a REST API using FastAPI that accepts uploaded images and returns prediction results in JSON format.
- Containerize the application using Docker to ensure reproducible deployment across different computing environments.
- Validate the complete prediction pipeline using representative sample images from the test dataset.

4. Dataset Description

This project uses the Street View House Numbers (SVHN) dataset, one of the most widely used benchmark datasets for image classification. The dataset consists of cropped color images extracted from Google Street View photographs, where each image contains a single digit.

Unlike handwritten digit datasets such as MNIST, the SVHN dataset contains naturally occurring images captured in outdoor environments. Variations in lighting, background objects, viewing angle, digit size, and image quality make the classification task significantly more challenging and representative of real-world computer vision applications.

Each image was converted to grayscale during preprocessing, producing a 32 × 32 pixel input image suitable for training the convolutional neural network.

Table 1 summarizes the number of images allocated to the training, validation, and testing datasets.

Table 1: Dataset Split Summary

Dataset Split	**Images**
Training	42,000
Validation	60,000
Testing	18,000

Table 2 summarizes the primary characteristics of the dataset used in this project.

Table 2: SVHN Dataset Characteristics

Property	**Value**
Image Size	32 × 32 pixels
Channels	Grayscale
Classes	10 (digits 0–9)
Data Type	Images
Learning Task	Multi-class Classification

5. Exploratory Data Analysis

Before training the convolutional neural network, the dataset was explored to better understand its composition, class distribution, and visual characteristics. Exploratory Data Analysis (EDA) helps verify data quality, identify potential class imbalance, and ensure that preprocessing decisions are appropriate before model development.

The analysis focused on answering the following questions:

- What do the digit images look like?
- Is the dataset balanced across all digit classes?
- Are there any obvious anomalies or quality issues?
- What preprocessing steps are required before training?

5.1. Representative Digit Images

Figure 1 presents one representative image from each of the ten digit classes. The images demonstrate the variation in brightness, contrast, stroke thickness, and visual appearance found in the dataset.

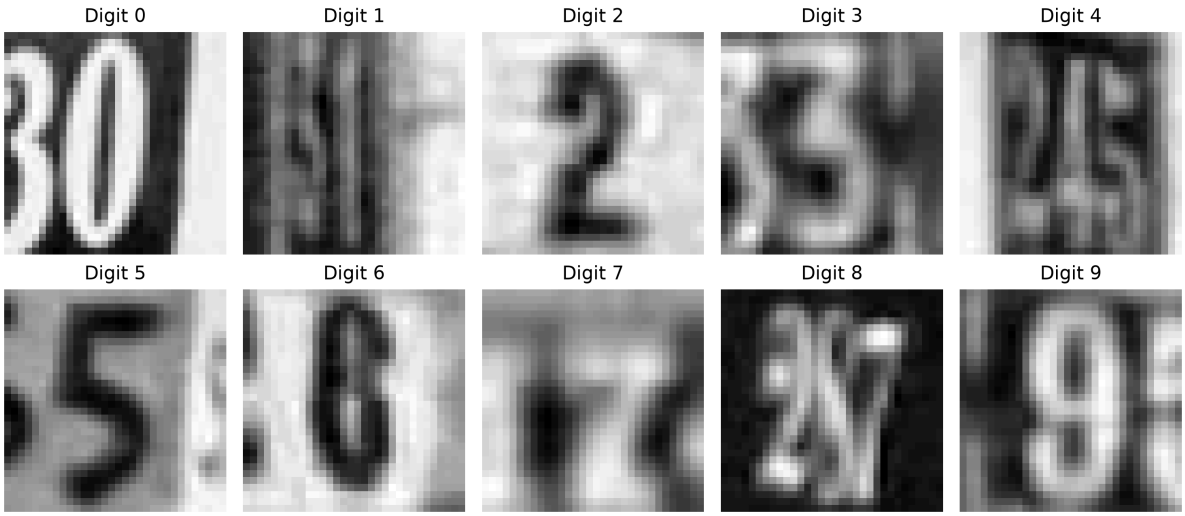

Figure 1: Representative images for the ten digit classes in the SVHN dataset.

As shown in Figure 1, the images contain naturally occurring variations that make the classification problem more challenging than recognizing clean, standardized handwritten digits.

5.2. Class Distribution

A balanced class distribution is important because it reduces the risk that the model will favor digits that appear more frequently during training. Figure 2 shows the number of training images available for each digit class.

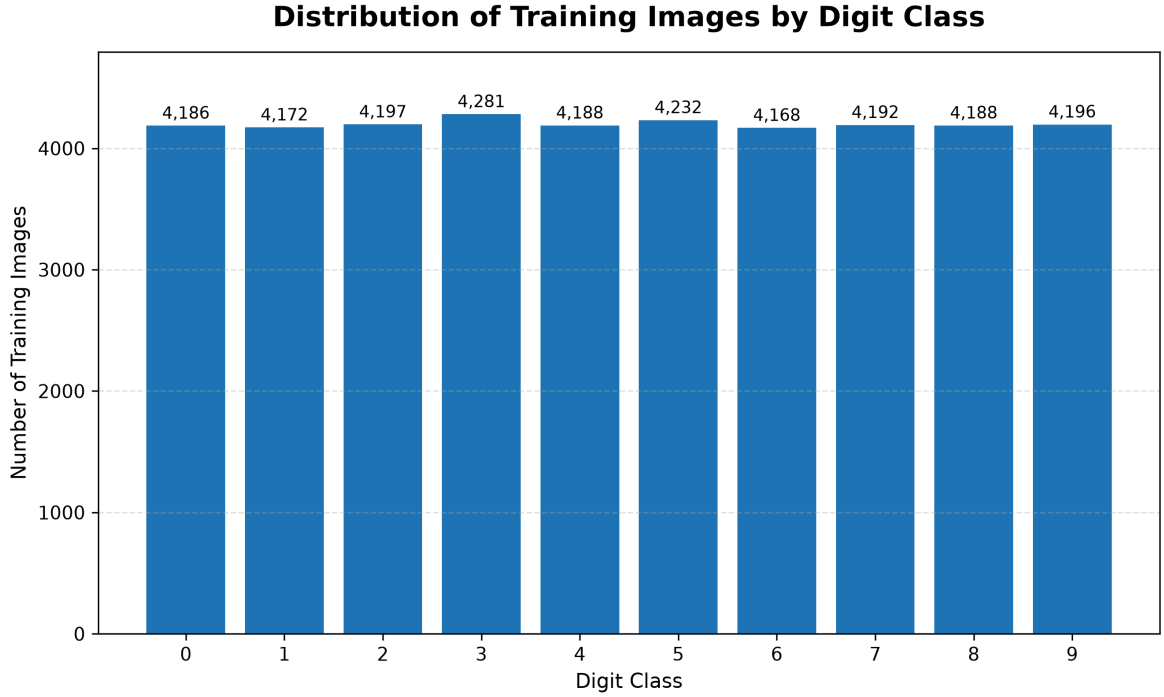

Figure 2: Distribution of training images across the ten SVHN digit classes.

Figure 2 shows that the training dataset is well balanced. Each digit class contains approximately 4,200 images, with only minor variation between the least represented and most represented classes. The smallest class contains 4,168 images, while the largest contains 4,281 images, a difference of only 113 samples.

Because no class is substantially underrepresented, class weighting or oversampling was not required for model training. This balanced distribution also makes overall accuracy a more informative evaluation metric, although precision, recall, F1-score, and the confusion matrix are still used to assess class-level performance.

5.3. Key EDA Observations

The exploratory analysis produced the following findings:

- The dataset contains ten clearly defined target classes representing the digits 0 through 9.
- All images have consistent dimensions of 32 × 32 pixels, which simplifies batching and model input preparation.
- The training data is evenly distributed across the ten digit classes, so no major class-imbalance treatment was necessary.
- The images contain realistic variation in brightness, contrast, stroke width, orientation, and surrounding visual context.
- Some images contain portions of neighboring digits or background details, making the task more challenging than classification using clean handwritten digit datasets.
- Pixel-value normalization is required before training so that the neural network receives inputs on a consistent numerical scale.

6. Data Preprocessing

The raw SVHN arrays were transformed into a representation suitable for convolutional neural-network training. The preprocessing workflow included reshaping the image arrays, normalizing pixel values, and encoding the target labels.

6.1. CNN Input Shape

The original grayscale images were stored as two-dimensional arrays with dimensions 32 × 32 pixels. Prior to training, a channel dimension was added to each image so that every sample had the shape **(32, 32, 1)**, matching the input format expected by the convolutional neural network.

The added final dimension represents the single grayscale channel. As a result, the complete training dataset is represented as a four-dimensional tensor with the shape **(42,000, 32, 32, 1)**.

6.2. Pixel-Value Normalization

Neural networks generally train more reliably when input values are placed on a consistent numerical scale. The original image pixels were represented using values in the range 0–255. Each pixel was divided by 255 so that the resulting values fell within the range 0.0–1.0.

Figure 3 compares the same image before and after normalization. Although the two images appear visually identical, the numerical representation of every pixel has changed.

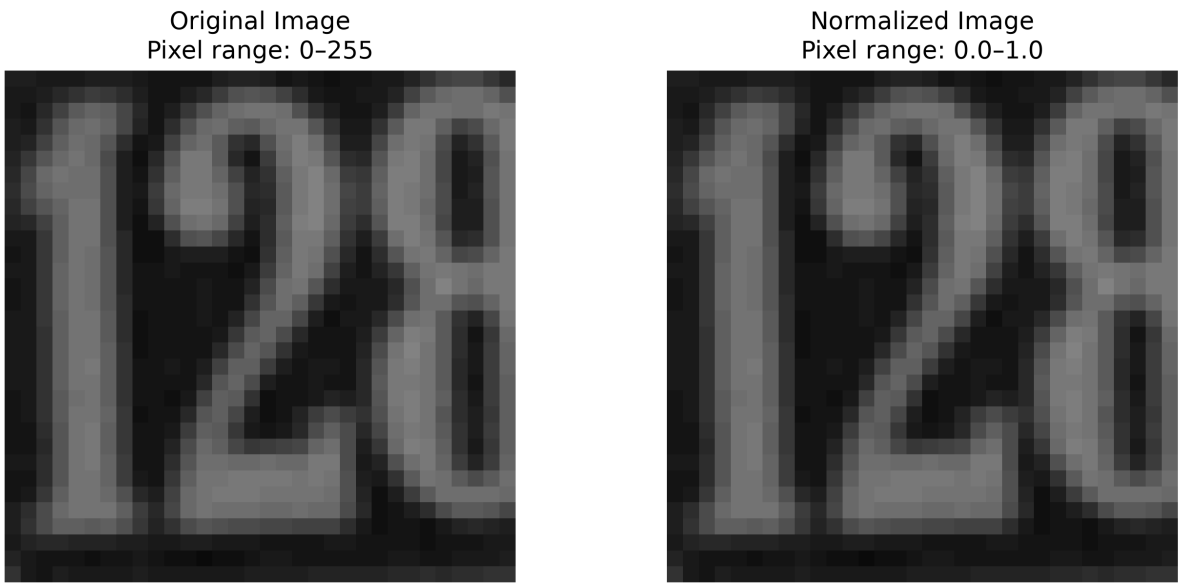

Figure 3: Comparison of an SVHN image before and after pixel-value normalization.

The original image uses pixel values in the range 0–255, while the normalized image uses values in the range 0.0–1.0. Normalizing the input improves numerical stability during gradient-based optimization while preserving the visual information contained in the image.

6.3. Pixel-Value Distribution

While Figure 3 illustrates the effect of normalization on a single image, it does not show how the transformation affects the dataset as a whole. To better understand the preprocessing step, the distribution of pixel values was analyzed before and after normalization.

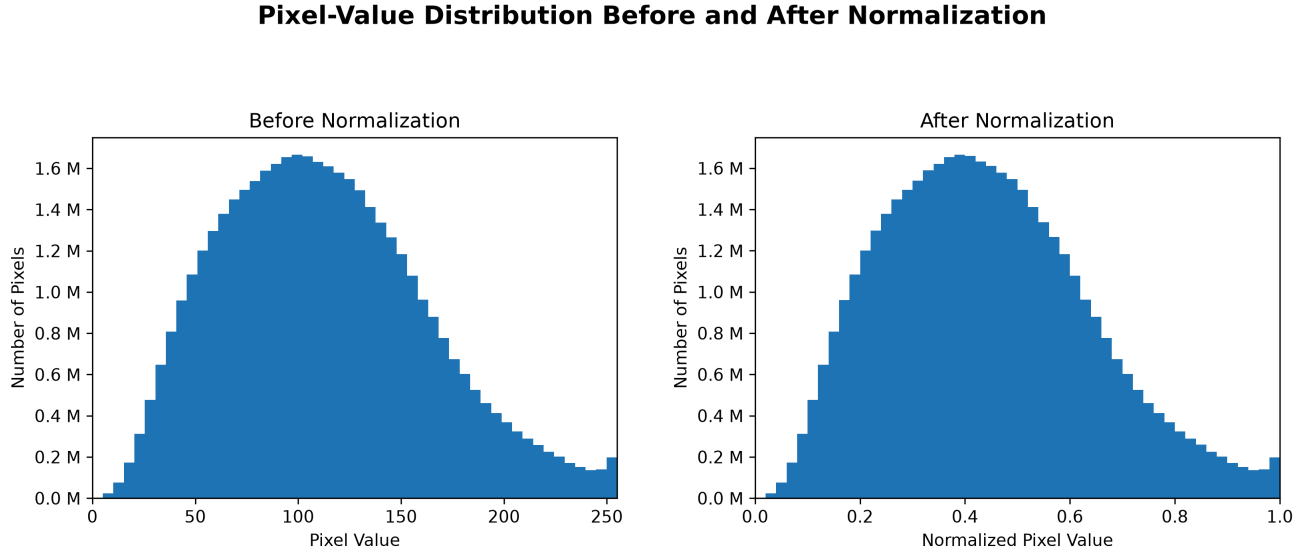

Figure 4: Distribution of pixel values before and after normalization across the complete SVHN training dataset.

Figure 4 demonstrates that normalization preserves the overall shape of the pixel-value distribution. The transformation changes only the numerical scale, mapping the original range of 0–255 to 0.0–1.0 through a linear scaling operation.

The distribution is concentrated around mid-range pixel intensities rather than being uniformly distributed across the available range. Because normalization is a linear transformation, the relative frequency of each pixel intensity is preserved while producing values that are more suitable for gradient-based optimization during model training.

6.4. Target Encoding

The image preprocessing steps described in the previous sections prepared the input data for the convolutional neural network. The target labels also required preprocessing before model training could begin.

Each training image is associated with one of the ten digit classes (0 through 9). Although these labels are stored as integer values, the CNN produces a probability for every output class rather than a single integer prediction. Consequently, the labels were converted into one-hot encoded vectors containing 10 positions.

For example, the digit label 3 becomes:

```
[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]
```

Each position in the vector corresponds to one output neuron in the softmax classification layer. The correct class is represented by a value of 1, while all remaining positions contain 0.

This representation enables the network to learn a probability distribution over all digit classes while using categorical cross-entropy as the training loss function.

6.5. Training–Serving Consistency

The preprocessing techniques described throughout this chapter prepare the training data for the convolutional neural network. These same transformations must also be applied during deployment so that the model receives data in the same format used during training.

For every image submitted to the FastAPI prediction endpoint, the preprocessing pipeline performs the following operations:

- Convert the image to grayscale.
- Resize the image to 32 × 32 pixels.
- Normalize pixel values from 0–255 to 0.0–1.0.
- Add the single-channel dimension required by the CNN.
- Reshape the image into the batch format expected by the model.

By sharing identical preprocessing logic between the training pipeline and the deployment pipeline, the project avoids **training-serving skew**, a condition in which a deployed model receives data that has been prepared differently from the data used during training.

Maintaining a consistent preprocessing pipeline ensures that every prediction is generated from images with the expected dimensions, numerical scale, channel configuration, and tensor structure. This consistency improves reliability and helps the deployed model reproduce the performance observed during model evaluation.

7. CNN Architecture

The convolutional neural network was designed to progressively transform the preprocessed grayscale input images into increasingly abstract visual features before producing a probability distribution across the ten digit classes.

The architecture consists of two feature-extraction blocks followed by a fully connected classification head. The convolutional blocks learn spatial patterns from the input images, while the classification head converts the learned feature representation into the final digit prediction.

7.1. High-Level Architecture

Figure 5 summarizes the major stages of the trained convolutional neural network. The model accepts a single grayscale image with dimensions 32 × 32 × 1 and passes it through two successive feature-extraction blocks before reaching the classification head.

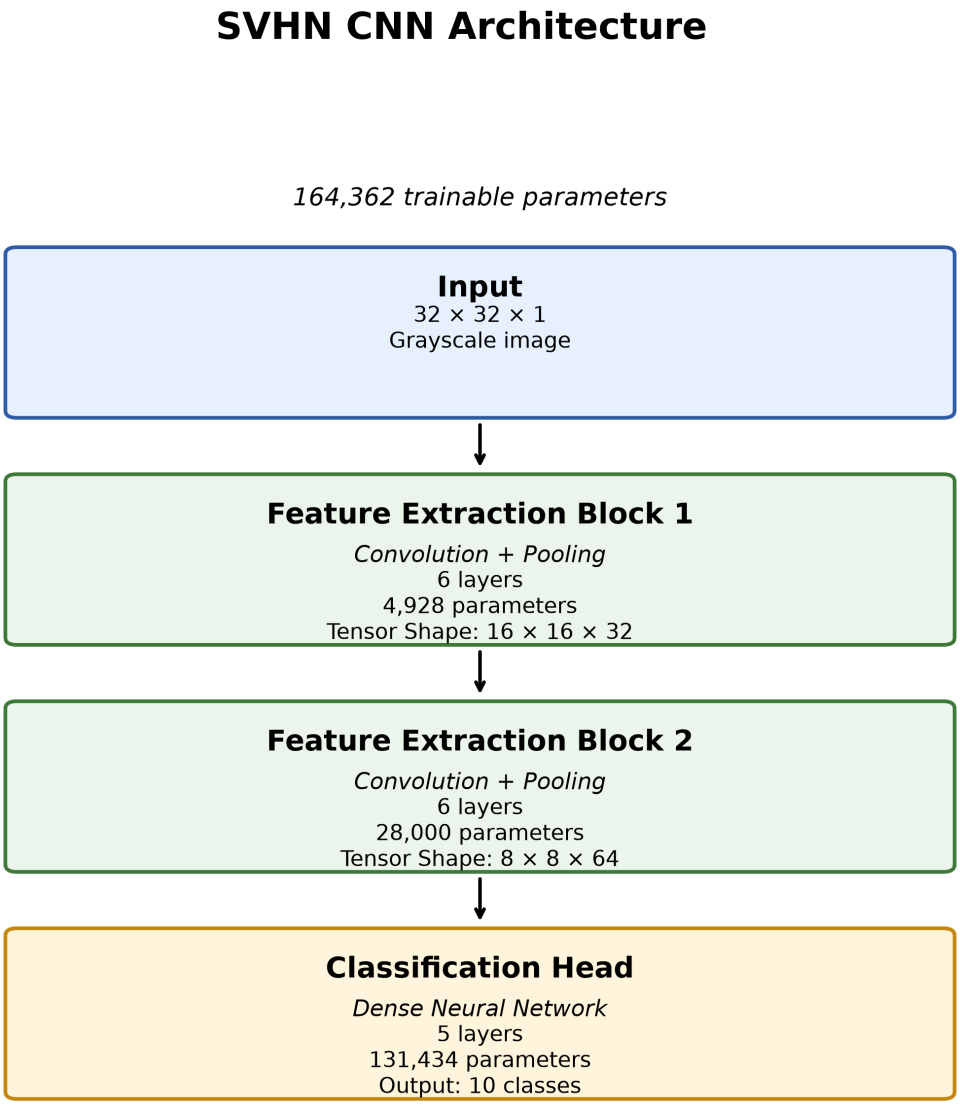

Figure 5: High-level architecture of the trained SVHN convolutional neural network.

The first feature-extraction block operates on the original 32 × 32 spatial representation and produces 32 feature maps. A max-pooling operation then reduces the spatial dimensions to 16 × 16.

The second feature-extraction block increases the learned representation to 64 feature maps while a second pooling operation reduces the spatial dimensions to 8 × 8. This pattern allows the CNN to trade spatial resolution for increasingly rich learned features.

The resulting feature maps are passed to the classification head, where they are flattened into a one-dimensional representation and processed by a dense neural-network layer. The final softmax layer produces one probability for each of the ten digit classes.

7.2. Detailed Layer Architecture

While Figure 5 presents the model at the component level, Figure 6 shows the complete layer-by-layer implementation. The detailed view preserves the same three architectural groups while exposing the transformations performed inside each block.

The first feature-extraction block contains two convolutional layers with LeakyReLU activation, followed by max pooling and batch normalization. The second block repeats the same general pattern while increasing the number of learned feature maps.

The classification head converts the final $8 \times 8 \times 64$ feature-map tensor into a 4,096 element feature vector. A dense layer reduces this representation to 32 learned features, followed by LeakyReLU activation and dropout regularization. The final dense layer contains 10 softmax outputs corresponding to the digit classes 0 through 9.

Figure 6: Detailed layer-by-layer architecture of the trained SVHN convolutional neural network.

7.3. Model Capacity

The trained CNN contains 164,362 parameters distributed across 17 layers, resulting in a relatively compact architecture for an image classification model. Most of the model capacity is concentrated in the dense classification layer after the convolutional feature maps are flattened.

The convolutional layers contain substantially fewer parameters because their filters are shared across spatial locations. In contrast, the first dense layer connects every element of the flattened feature representation to each of its output neurons, resulting in the largest parameter contribution within the network.

This distribution of parameters reflects the different responsibilities of the two portions of the model: the convolutional blocks extract spatial features, while the dense layers combine those learned features to perform final classification.

7.4. Architecture Summary

The table below summarizes the primary structural characteristics of the trained convolutional neural network.

Table 3: Summary of the trained CNN architecture and model capacity.

Architecture Property	Value
Input Shape	32 × 32 × 1
Total Layers	17
Total Parameters	164,362
Final Feature Shape	8 × 8 × 64
Flattened Features	4,096
Hidden Dense Units	32
Output Classes	10
Convolution Kernel	3 × 3
Dropout Rate	0.5

7.5. Design Decisions

Several architectural choices were used to improve feature learning, optimization stability, and generalization.

The convolutional layers use 3 × 3 kernels with same padding, allowing the network to learn local spatial patterns while preserving feature-map dimensions until pooling is applied. The number of feature maps increases as the network becomes deeper, enabling progressively richer visual representations.

LeakyReLU activation is used throughout the hidden layers so that small negative activations can continue to propagate rather than being forced entirely to zero. Max-pooling layers reduce spatial dimensionality and computational cost, while batch normalization helps maintain stable feature distributions during training.

A dropout layer is included in the classification head to reduce reliance on individual neurons and improve generalization. Finally, the ten-unit softmax output layer converts the network output into a probability distribution across the ten SVHN digit classes.